

Kompresa: Slovníkové metody

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

05.06.2026

CC BY-NC-SA 4.0

Slovníkové metody

Od statistických k slovníkovým metodám

Od statistických k slovníkovým metodám

Huffman a aritmetické kódování pracují se **statistikou jednotlivých symbolů**

Od statistických k slovníkovým metodám

Huffman a aritmetické kódování pracují se **statistikou jednotlivých symbolů**

Ale v reálných datech se opakují celé **sekvence**:

Od statistických k slovníkovým metodám

Huffman a aritmetické kódování pracují se **statistikou jednotlivých symbolů**

Ale v reálných datech se opakují celé **sekvence**:

the cat sat on the mat on the flat

Od statistických k slovníkovým metodám

Huffman a aritmetické kódování pracují se **statistikou jednotlivých symbolů**

Ale v reálných datech se opakují celé **sekvence**:

the cat sat on the mat on the flat

the se opakuje 3 ×, on 2 ×, at 4 ×

Od statistických k slovníkovým metodám

Huffman a aritmetické kódování pracují se **statistikou jednotlivých symbolů**

Ale v reálných datech se opakují celé **sekvence**:

the cat sat on the mat on the flat

the se opakuje 3 ×, on 2 ×, at 4 × – co kdybychom opakující se sekvence **nahradili referencemi**?

Od statistických k slovníkovým metodám

Jakou analogii si můžete představit?

Od statistických k slovníkovým metodám

Jakou analogii si můžete představit?

Představte si, že píšete poznámky a místo opakování celé věty napíšete

Od statistických k slovníkovým metodám

Jakou analogii si můžete představit?

Představte si, že píšete poznámky a místo opakování celé věty napíšete „viz strana 3, třetí odstavec“

Od statistických k slovníkovým metodám

Jakou analogii si můžete představit?

Představte si, že píšete poznámky a místo opakování celé věty napíšete „viz strana 3, třetí odstavec“

Přesně tak fungují **slovníkové metody**

Od statistických k slovníkovým metodám

Jakou analogii si můžete představit?

Představte si, že píšete poznámky a místo opakování celé věty napíšete „viz strana 3, třetí odstavec“

Přesně tak fungují **slovníkové metody** – nahrazují opakující se vzory **odkazem** na místo, kde se již vyskytly

LZ77

Posuvné okno

Posuvné okno

Abraham Lempel a Jacob Ziv (1977) navrhli metodu založenou na **posuvném okně** (*sliding window*)

Posuvné okno

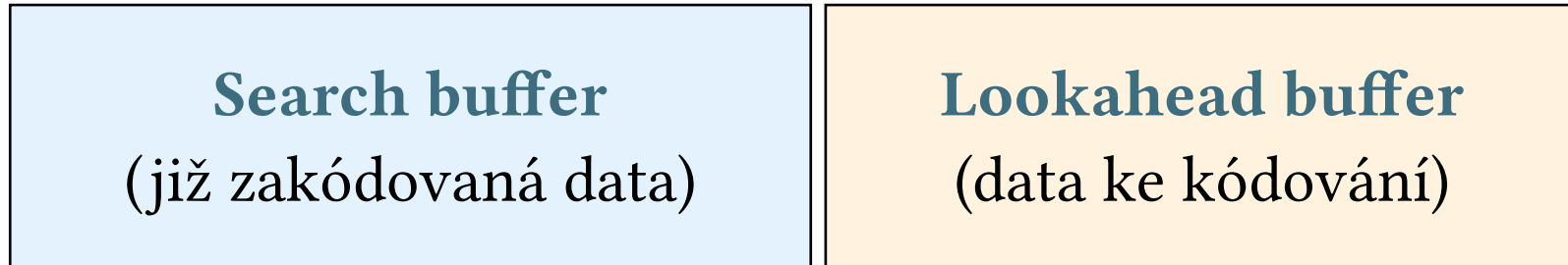
Abraham Lempel a Jacob Ziv (1977) navrhli metodu založenou na **posuvném okně** (*sliding window*)

Search buffer
(již zakódovaná data)

Lookahead buffer
(data ke kódování)

Posuvné okno

Abraham Lempel a Jacob Ziv (1977) navrhli metodu založenou na **posuvném okně** (*sliding window*)



V search bufferu hledáme **nejdelší shodu** s tím, co začíná v lookahead bufferu

Posuvné okno

Každý krok zapíšeme jako trojici:

Posuvné okno

Každý krok zapíšeme jako trojici: (offset, length, next_char)

Posuvné okno

Každý krok zapíšeme jako trojici: (offset, length, next_char)

- **offset** – jak daleko zpět v search bufferu začíná shoda

Posuvné okno

Každý krok zapíšeme jako trojici: (offset, length, next_char)

- **offset** – jak daleko zpět v search bufferu začíná shoda
- **length** – délka shody

Posuvné okno

Každý krok zapíšeme jako trojici: (offset, length, next_char)

- **offset** – jak daleko zpět v search bufferu začíná shoda
- **length** – délka shody
- **next_char** – první znak **po** shodě

Posuvné okno

Každý krok zapíšeme jako trojici: (offset, length, next_char)

- **offset** – jak daleko zpět v search bufferu začíná shoda
- **length** – délka shody
- **next_char** – první znak **po** shodě

Pokud žádná shoda neexistuje →

Posuvné okno

Každý krok zapíšeme jako trojici: (offset, length, next_char)

- **offset** – jak daleko zpět v search bufferu začíná shoda
- **length** – délka shody
- **next_char** – první znak **po** shodě

Pokud žádná shoda neexistuje $\rightarrow (0, 0, \text{char})$ – literál

Posuvné okno

Každý krok zapíšeme jako trojici: (offset, length, next_char)

- **offset** – jak daleko zpět v search bufferu začíná shoda
- **length** – délka shody
- **next_char** – první znak **po** shodě

Pokud žádná shoda neexistuje $\rightarrow (0, 0, \text{char})$ – literál

Pokud shoda sahá až na konec vstupu a žádný další znak neexistuje, použijeme \$ jako **ukončovací značku**

Kódování LZ77

Zakódujme ABCABCABC:

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1				

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–			

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .		

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2				

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A			

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .		

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3				

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB			

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC		

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	C je nový symbol

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	C je nový symbol
4				

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	C je nový symbol
4	ABC			

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	C je nový symbol
4	ABC	ABCABC		

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	C je nový symbol
4	ABC	ABCABC	(3,3,A)	

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	C je nový symbol
4	ABC	ABCABC	(3,3,A)	ABC: shoda 3 zpět

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	C je nový symbol
4	ABC	ABCABC	(3,3,A)	ABC: shoda 3 zpět
5				

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	C je nový symbol
4	ABC	ABCABC	(3,3,A)	ABC: shoda 3 zpět
5	ABCABCA			

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	C je nový symbol
4	ABC	ABCABC	(3,3,A)	ABC: shoda 3 zpět
5	ABCABCA	BC		

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	C je nový symbol
4	ABC	ABCABC	(3,3,A)	ABC: shoda 3 zpět
5	ABCABCA	BC	(3,2,\$)	

Kódování LZ77

Zakódujme ABCABCABC:

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	ABCABC . . .	(0,0,A)	A je nový symbol
2	A	BCABCA . . .	(0,0,B)	B je nový symbol
3	AB	CABCABC	(0,0,C)	C je nový symbol
4	ABC	ABCABC	(3,3,A)	ABC: shoda 3 zpět
5	ABCABCA	BC	(3,2,\$)	BC: shoda 3 zpět

Kódování LZ77

Vstup: ABCABCABC (9 znaků)

Kódování LZ77

Vstup: ABCABCABC (9 znaků)

Výstup: 5 trojic: $(0, 0, A)$ $(0, 0, B)$ $(0, 0, C)$ $(3, 3, A)$ $(3, 2, \$)$

Kódování LZ77

Vstup: ABCABCABC (9 znaků)

Výstup: 5 trojic: $(0, 0, A)$ $(0, 0, B)$ $(0, 0, C)$ $(3, 3, A)$ $(3, 2, \$)$

Klíčový moment: v kroku 4 jsme **3 znaky** nahradili jedním odkazem

Kódování LZ77

Vstup: ABCABCABC (9 znaků)

Výstup: 5 trojic: $(0, 0, A)$ $(0, 0, B)$ $(0, 0, C)$ $(3, 3, A)$ $(3, 2, \$)$

Klíčový moment: v kroku 4 jsme **3 znaky** nahradili jedním odkazem

Čím delší shody najdeme, tím lepší komprese

Kódování LZ77

Vstup: ABCABCABC (9 znaků)

Výstup: 5 trojic: (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$)

Klíčový moment: v kroku 4 jsme **3 znaky** nahradili jedním odkazem

Čím delší shody najdeme, tím lepší komprese – proto záleží na **velikosti okna**

Dekódování LZ77

Jak dekodovat?

Dekódování LZ77

Jak dekodovat?

Dekódování je **jednodušší** než kódování – stačí následovat reference

Dekódování LZ77

Jak dekodovat?

Dekódování je **jednodušší** než kódování – stačí následovat reference

Pro každou trojici (offset, length, next_char):

Dekódování LZ77

Jak dekodovat?

Dekódování je **jednodušší** než kódování – stačí následovat reference

Pro každou trojici (`offset`, `length`, `next_char`):

1. Pokud `length > 0`: zkopíruj `length` znaků z pozice `offset` zpět

Dekódování LZ77

Jak dekodovat?

Dekódování je **jednodušší** než kódování – stačí následovat reference

Pro každou trojici (`offset`, `length`, `next_char`):

1. Pokud `length > 0`: zkopíruj `length` znaků z pozice `offset` zpět
2. Připoj `next_char` na konec výstupu

Dekódování LZ77

Dekódujme $(0,0,A)$ $(0,0,B)$ $(0,0,C)$ $(3,3,A)$ $(3,2,\$)$:

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)		

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)		

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	AB

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	AB
3	(0,0,C)		

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	AB
3	(0,0,C)	literál C	

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	AB
3	(0,0,C)	literál C	ABC

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	AB
3	(0,0,C)	literál C	ABC
4	(3,3,A)		

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	AB
3	(0,0,C)	literál C	ABC
4	(3,3,A)	3 zpět od pozice 3 = pozice 0, kopíruj 3 + A	

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	AB
3	(0,0,C)	literál C	ABC
4	(3,3,A)	3 zpět od pozice 3 = pozice 0, kopíruj 3 + A	ABCABCA

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	AB
3	(0,0,C)	literál C	ABC
4	(3,3,A)	3 zpět od pozice 3 = pozice 0, kopíruj 3 + A	ABCABCA
5	(3,2,\$)		

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	AB
3	(0,0,C)	literál C	ABC
4	(3,3,A)	3 zpět od pozice 3 = pozice 0, kopíruj 3 + A	ABCABCA
5	(3,2,\$)	3 zpět od pozice 7 = pozice 4, kopíruj 2	

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	AB
3	(0,0,C)	literál C	ABC
4	(3,3,A)	3 zpět od pozice 3 = pozice 0, kopíruj 3 + A	ABCABCA
5	(3,2,\$)	3 zpět od pozice 7 = pozice 4, kopíruj 2	ABCABCABC

Dekódování LZ77

Dekódujme (0,0,A) (0,0,B) (0,0,C) (3,3,A) (3,2,\$):

Krok	Kód	Akce	Výstup
1	(0,0,A)	literál A	A
2	(0,0,B)	literál B	AB
3	(0,0,C)	literál C	ABC
4	(3,3,A)	3 zpět od pozice 3 = pozice 0, kopíruj 3 + A	ABCABCA
5	(3,2,\$)	3 zpět od pozice 7 = pozice 4, kopíruj 2	ABCABCABC

Výsledek: ABCABCABC ✓

Délka shody přes hranici bufferu

Co když je `length` větší než `offset`?

Délka shody přes hranici bufferu

Co když je `length` větší než `offset`?

Zakódujme `AAAAAA` ($6 \times A$):

Délka shody přes hranici bufferu

Co když je `length` větší než `offset`?

Zakódujme AAAAAA ($6 \times A$):

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1				

Délka shody přes hranici bufferu

Co když je `length` větší než `offset`?

Zakódujme `AAAAAA` ($6 \times A$):

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–			

Délka shody přes hranici bufferu

Co když je `length` větší než `offset`?

Zakódujme `AAAAAA` ($6 \times A$):

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	AAAAAA		

Délka shody přes hranici bufferu

Co když je **length** větší než **offset**?

Zakódujme AAAAAA ($6 \times A$):

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	AAAAAA	(0,0,A)	

Délka shody přes hranici bufferu

Co když je `length` větší než `offset`?

Zakódujme `AAAAAA` ($6 \times A$):

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	AAAAAA	(0,0,A)	A je nový

Délka shody přes hranici bufferu

Co když je **length** větší než **offset**?

Zakódujme AAAAAA ($6 \times A$):

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	AAAAAA	(0,0,A)	A je nový
2				

Délka shody přes hranici bufferu

Co když je `length` větší než `offset`?

Zakódujme `AAAAAA` ($6 \times A$):

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	AAAAAA	(0,0,A)	A je nový
2	A			

Délka shody přes hranici bufferu

Co když je **length** větší než **offset**?

Zakódujme AAAAAA ($6 \times A$):

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	AAAAAA	(0,0,A)	A je nový
2	A	AAAAA		

Délka shody přes hranici bufferu

Co když je **length** větší než **offset**?

Zakódujme AAAAAA ($6 \times A$):

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	AAAAAA	(0,0,A)	A je nový
2	A	AAAAA	(1,5,\$)	

Délka shody přes hranici bufferu

Co když je **length** větší než **offset**?

Zakódujme AAAAAA ($6 \times A$):

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	AAAAAA	(0,0,A)	A je nový
2	A	AAAAA	(1,5,\$)	shoda 1 zpět, délka 5

Délka shody přes hranici bufferu

Co když je **length** větší než **offset**?

Zakódujme AAAAAA ($6 \times A$):

Krok	Search buffer	Lookahead	Kód	Vysvětlení
1	–	AAAAAA	(0,0,A)	A je nový
2	A	AAAAA	(1,5,\$)	shoda 1 zpět, délka 5

Search buffer má **1 znak**, ale délka shody je **5** – jak je to možné?

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1,5,\$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1		

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 → A	

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 → A	AA

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 → A	AA
2		

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	AA
2	pozice 1 $\rightarrow$ A	

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	AA
2	pozice 1 $\rightarrow$ A	AAA

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	AA
2	pozice 1 $\rightarrow$ A	AAA
3		

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	AA
2	pozice 1 $\rightarrow$ A	AAA
3	pozice 2 $\rightarrow$ A	

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	AA
2	pozice 1 $\rightarrow$ A	AAA
3	pozice 2 $\rightarrow$ A	AAAA

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	AA
2	pozice 1 $\rightarrow$ A	AAA
3	pozice 2 $\rightarrow$ A	AAAA
4		

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1,5,\$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	AA
2	pozice 1 $\rightarrow$ A	AAA
3	pozice 2 $\rightarrow$ A	AAAA
4	pozice 3 $\rightarrow$ A	

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	AA
2	pozice 1 $\rightarrow$ A	AAA
3	pozice 2 $\rightarrow$ A	AAAA
4	pozice 3 $\rightarrow$ A	AAAAA

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1,5,\$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	AA
2	pozice 1 $\rightarrow$ A	AAA
3	pozice 2 $\rightarrow$ A	AAAA
4	pozice 3 $\rightarrow$ A	AAAAA
5		

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	AA
2	pozice 1 $\rightarrow$ A	AAA
3	pozice 2 $\rightarrow$ A	AAAA
4	pozice 3 $\rightarrow$ A	AAAAA
5	pozice 4 $\rightarrow$ A	

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1,5,\$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	AA
2	pozice 1 $\rightarrow$ A	AAA
3	pozice 2 $\rightarrow$ A	AAAA
4	pozice 3 $\rightarrow$ A	AAAAA
5	pozice 4 $\rightarrow$ A	AAAAAA

Délka shody přes hranici bufferu

Dekodér kopíruje **znak po znaku** – každý zkopírovaný znak je hned k dispozici:

Dekódujme (1, 5, \$) (výstup zatím: A):

Kopie	Zdroj	Výstup
1	pozice 0 → A	AA
2	pozice 1 → A	AAA
3	pozice 2 → A	AAAA
4	pozice 3 → A	AAAAA
5	pozice 4 → A	AAAAAA

LZ77 tak umí fungovat i jako **RLE** – opakování jednoho znaku je jen (1, N, \$)

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

(0,0,A) (0,0,B) (2,4,\$) – offset 2, délka 4

Dekódování (2,4,\$) (výstup zatím: AB):

Kopie	Zdroj	Výstup
1		

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 → A	

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	ABA

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	ABA
2		

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	ABA
2	pozice 1 $\rightarrow$ B	

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	ABA
2	pozice 1 $\rightarrow$ B	ABAB

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	ABA
2	pozice 1 $\rightarrow$ B	ABAB
3		

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	ABA
2	pozice 1 $\rightarrow$ B	ABAB
3	pozice 2 $\rightarrow$ A	

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	ABA
2	pozice 1 $\rightarrow$ B	ABAB
3	pozice 2 $\rightarrow$ A	ABABA

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	ABA
2	pozice 1 $\rightarrow$ B	ABAB
3	pozice 2 $\rightarrow$ A	ABABA
4		

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	ABA
2	pozice 1 $\rightarrow$ B	ABAB
3	pozice 2 $\rightarrow$ A	ABABA
4	pozice 3 $\rightarrow$ B	

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0, 0, A)$ $(0, 0, B)$ $(2, 4, \$)$ – offset 2, délka 4

Dekódování $(2, 4, \$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	ABA
2	pozice 1 $\rightarrow$ B	ABAB
3	pozice 2 $\rightarrow$ A	ABABA
4	pozice 3 $\rightarrow$ B	ABABAB

Délka shody přes hranici bufferu

Funguje to i pro víceznakové vzory – zakódujme ABABAB:

$(0,0,A)$ $(0,0,B)$ $(2,4,\$)$ – offset 2, délka 4

Dekódování $(2,4,\$)$ (výstup zatím: AB):

Kopie	Zdroj	Výstup
1	pozice 0 $\rightarrow$ A	ABA
2	pozice 1 $\rightarrow$ B	ABAB
3	pozice 2 $\rightarrow$ A	ABABA
4	pozice 3 $\rightarrow$ B	ABABAB

Offset udává **periodu vzoru** – $(1,N,\$)$ opakuje 1 znak, $(2,N,\$)$ dva znaky, $(3,N,\$)$ tři, ...

Velikost okna

Jak velké má být posuvné okno?

Velikost okna

Jak velké má být posuvné okno?

Větší okno →

Velikost okna

Jak velké má být posuvné okno?

Větší okno → více příležitostí najít shody →

Velikost okna

Jak velké má být posuvné okno?

Větší okno → více příležitostí najít shody → lepší komprese

Velikost okna

Jak velké má být posuvné okno?

Větší okno → více příležitostí najít shody → lepší komprese

Ale také →

Velikost okna

Jak velké má být posuvné okno?

Větší okno → více příležitostí najít shody → lepší komprese

Ale také → delší hledání →

Velikost okna

Jak velké má být posuvné okno?

Větší okno → více příležitostí najít shody → lepší komprese

Ale také → delší hledání → pomalejší komprese

Velikost okna

Jak velké má být posuvné okno?

Větší okno → více příležitostí najít shody → lepší komprese

Ale také → delší hledání → pomalejší komprese

Formát	Velikost okna	Poznámka
gzip (deflate)	32 KB	kompromis rychlost/komprese
zstd	až 128 MB	rychlejší díky hash tabulkám
LZMA (7z)	až 4 GB	nejlepší komprese, pomalé

Kde se LZ77 používá?

Kde se LZ77 používá?

- `gzip`

Kde se LZ77 používá?

- **gzip** – standardní komprese na Linuxu

Kde se LZ77 používá?

- **gzip** – standardní komprese na Linuxu
- **Deflate**

Kde se LZ77 používá?

- **gzip** – standardní komprese na Linuxu
- **Deflate** – kombinace LZ77 + Huffman

Kde se LZ77 používá?

- **gzip** – standardní komprese na Linuxu
- **Deflate** – kombinace LZ77 + Huffman (základ **ZIP**, **PNG**, HTTP komprese)

Kde se LZ77 používá?

- **gzip** – standardní komprese na Linuxu
- **Deflate** – kombinace LZ77 + Huffman (základ **ZIP**, **PNG**, HTTP komprese)
- **zstd** (Zstandard)

Kde se LZ77 používá?

- **gzip** – standardní komprese na Linuxu
- **Deflate** – kombinace LZ77 + Huffman (základ **ZIP**, **PNG**, HTTP komprese)
- **zstd** (Zstandard) – moderní rychlá komprese od Facebooku

Kde se LZ77 používá?

- **gzip** – standardní komprese na Linuxu
- **Deflate** – kombinace LZ77 + Huffman (základ **ZIP**, **PNG**, HTTP komprese)
- **zstd** (Zstandard) – moderní rychlá komprese od Facebooku
- **LZMA**

Kde se LZ77 používá?

- **gzip** – standardní komprese na Linuxu
- **Deflate** – kombinace LZ77 + Huffman (základ **ZIP**, **PNG**, HTTP komprese)
- **zstd** (Zstandard) – moderní rychlá komprese od Facebooku
- **LZMA** – základ formátu 7z, nejlepší kompresní poměr

Kde se LZ77 používá?

- **gzip** – standardní komprese na Linuxu
- **Deflate** – kombinace LZ77 + Huffman (základ **ZIP**, **PNG**, HTTP komprese)
- **zstd** (Zstandard) – moderní rychlá komprese od Facebooku
- **LZMA** – základ formátu 7z, nejlepší kompresní poměr

Deflate je pravděpodobně **nejpoužívanější** kompresní algoritmus na světě

LZ78

Od posuvného okna k explicitnému slovníku

Od posuvného okna k explicitnímu slovníku

LZ77 hledá shody v **posuvném okně**

Od posuvného okna k explicitnímu slovníku

LZ77 hledá shody v **posuvném okně** – to má omezení:

Od posuvného okna k explicitnímu slovníku

LZ77 hledá shody v **posuvném okně** – to má omezení:

- Okno má **omezenou velikost** – starší vzory se ztratí

Od posuvného okna k explicitnímu slovníku

LZ77 hledá shody v **posuvném okně** – to má omezení:

- Okno má **omezenou velikost** – starší vzory se ztratí
- Hledání nejdelší shody v okně může být **pomalé**

Od posuvného okna k explicitnímu slovníku

LZ77 hledá shody v **posuvném okně** – to má omezení:

- Okno má **omezenou velikost** – starší vzory se ztratí
- Hledání nejdelší shody v okně může být **pomalé**

Abraham Lempel a Jacob Ziv (1978) přišli s jiným přístupem:

Od posuvného okna k explicitnímu slovníku

LZ77 hledá shody v **posuvném okně** – to má omezení:

- Okno má **omezenou velikost** – starší vzory se ztratí
- Hledání nejdelší shody v okně může být **pomalé**

Abraham Lempel a Jacob Ziv (1978) přišli s jiným přístupem: místo posuvného okna budují **explicitní slovník**

Od posuvného okna k explicitnímu slovníku

Každý záznam ve slovníku dostane **číselný kód**

Od posuvného okna k explicitnímu slovníku

Každý záznam ve slovníku dostane **číselný kód**

Na výstupu zapisujeme **dvojice**:

Od posuvného okna k explicitnímu slovníku

Každý záznam ve slovníku dostane **číselný kód**

Na výstupu zapisujeme **dvojice**: (index, char)

Od posuvného okna k explicitnímu slovníku

Každý záznam ve slovníku dostane **číselný kód**

Na výstupu zapisujeme **dvojice**: (index, char)

- **index** – odkaz na existující záznam ve slovníku (0 = prázdný řetězec)

Od posuvného okna k explicitnímu slovníku

Každý záznam ve slovníku dostane **číselný kód**

Na výstupu zapisujeme **dvojice**: (index, char)

- **index** – odkaz na existující záznam ve slovníku (0 = prázdný řetězec)
- **char** – nový znak, který prodlužuje nalezenou sekvenci

Kódování LZ78

Kódování LZ78

Algoritmus:

Kódování LZ78

Algoritmus:

1. Hledej **nejdelší sekvenci** ze vstupu, která je ve slovníku

Kódování LZ78

Algoritmus:

1. Hledej **nejdelší sekvenci** ze vstupu, která je ve slovníku
2. Vypiš dvojici (kód nalezené sekvence, následující znak)

Kódování LZ78

Algoritmus:

1. Hledej **nejdelší sekvenci** ze vstupu, která je ve slovníku
2. Vypiš dvojici (kód nalezené sekvence, následující znak)
3. Přidej nalezenou sekvenci + následující znak do slovníku

Kódování LZ78

Algoritmus:

1. Hledej **nejdelší sekvenci** ze vstupu, která je ve slovníku
2. Vypiš dvojici (kód nalezené sekvence, následující znak)
3. Přidej nalezenou sekvenci + následující znak do slovníku
4. Opakuj od kroku 1

Kódování LZ78

Algoritmus:

1. Hledej **nejdelší sekvenci** ze vstupu, která je ve slovníku
2. Vypiš dvojici (kód nalezené sekvence, následující znak)
3. Přidej nalezenou sekvenci + následující znak do slovníku
4. Opakuj od kroku 1

Slovník začíná s jedním záznamem: **0 = prázdný řetězec**

Kódování LZ78

Zakódujme ABABAB:

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB			

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–		

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB			

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–		

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	přidej 2 = B

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A
2	B

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	přidej 2 = B

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A
2	B

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	přidej 2 = B
3	ABAB			

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A
2	B

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	přidej 2 = B
3	ABAB	A (kód 1)		

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A
2	B

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	přidej 2 = B
3	ABAB	A (kód 1)	(1, B)	

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A
2	B

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	přidej 2 = B
3	ABAB	A (kód 1)	(1, B)	přidej 3 = AB

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	přidej 2 = B
3	ABAB	A (kód 1)	(1, B)	přidej 3 = AB

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	přidej 2 = B
3	ABAB	A (kód 1)	(1, B)	přidej 3 = AB
4	AB			

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	přidej 2 = B
3	ABAB	A (kód 1)	(1, B)	přidej 3 = AB
4	AB	AB (kód 3)		

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	přidej 2 = B
3	ABAB	A (kód 1)	(1, B)	přidej 3 = AB
4	AB	AB (kód 3)	(3, \$)	

Kódování LZ78

Zakódujme ABABAB:

Slovník:

Kód	Záznam
0	" "
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Shoda	Výstup	Akce
1	ABABAB	–	(0, A)	přidej 1 = A
2	BABAB	–	(0, B)	přidej 2 = B
3	ABAB	A (kód 1)	(1, B)	přidej 3 = AB
4	AB	AB (kód 3)	(3, \$)	–

Vstup: 6 znaků → výstup: 4 dvojice

Kódování LZ78

Proč je krok 4 důležitý?

Kódování LZ78

Proč je krok 4 důležitý?

V kroku 4 jsme odeslali kód pro „AB“, který jsme přidali teprve v kroku 3

Kódování LZ78

Proč je krok 4 důležitý?

V kroku 4 jsme odeslali kód pro „AB“, který jsme přidali teprve v kroku 3

Slovník se **učí** vzory ze vstupních dat

Kódování LZ78

Proč je krok 4 důležitý?

V kroku 4 jsme odeslali kód pro „AB“, který jsme přidali teprve v kroku 3

Slovník se **učí** vzory ze vstupních dat a pak je **využívá** v dalším kódování

Kódování LZ78

Proč je krok 4 důležitý?

V kroku 4 jsme odeslali kód pro „AB“, který jsme přidali teprve v kroku 3

Slovník se **učí** vzory ze vstupních dat a pak je **využívá** v dalším kódování

Čím delší vstup s opakujícími se vzory, tím lépe slovníkové metody fungují

Dekódování LZ78

Dekódování LZ78

Dekodér si buduje **stejný slovník** jako kodér

Dekódování LZ78

Dekodér si buduje **stejný slovník** jako kodér – stačí následovat dvojice

Dekódování LZ78

Dekodér si buduje **stejný slovník** jako kodér – stačí následovat dvojice

Pro každou dvojici (index, char):

Dekódování LZ78

Dekodér si buduje **stejný slovník** jako kodér – stačí následovat dvojice

Pro každou dvojici (index, char):

1. Najdi sekvenci s kódem index ve slovníku

Dekódování LZ78

Dekodér si buduje **stejný slovník** jako kodér – stačí následovat dvojice

Pro každou dvojici (index, char):

1. Najdi sekvenci s kódem index ve slovníku
2. Připoj char na konec

Dekódování LZ78

Dekodér si buduje **stejný slovník** jako kodér – stačí následovat dvojice

Pro každou dvojici (index, char):

1. Najdi sekvenci s kódem index ve slovníku
2. Připoj char na konec
3. Přidej výsledek do slovníku jako nový záznam

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)		

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	A

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	A

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	A
2	(0, B)		

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	− + A = A	A
2	(0, B)	− + B = B	

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	− + A = A	A
2	(0, B)	− + B = B	AB

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A
2	B

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	A
2	(0, B)	$- + B = B$	AB

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A
2	B

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	A
2	(0, B)	$- + B = B$	AB
3	(1, B)		

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A
2	B

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	A
2	(0, B)	$- + B = B$	AB
3	(1, B)	$A + B = AB$	

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A
2	B

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	A
2	(0, B)	$- + B = B$	AB
3	(1, B)	$A + B = AB$	ABAB

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A
2	B
3	AB

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	A
2	(0, B)	$- + B = B$	AB
3	(1, B)	$A + B = AB$	ABAB

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A
2	B
3	AB

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	A
2	(0, B)	$- + B = B$	AB
3	(1, B)	$A + B = AB$	ABAB
4	(3, \$)		

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A
2	B
3	AB

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	A
2	(0, B)	$- + B = B$	AB
3	(1, B)	$A + B = AB$	ABAB
4	(3, \$)	AB	

Dekódování LZ78

Dekódujme (0,A) (0,B) (1,B) (3,\$):

Slovník:

Kód	Záznam
0	" "
1	A
2	B
3	AB

Dekódování:

Krok	Vstup	Akce	Výstup
1	(0, A)	$- + A = A$	A
2	(0, B)	$- + B = B$	AB
3	(1, B)	$A + B = AB$	ABAB
4	(3, \$)	AB	ABABAB

Výsledek: ABABAB ✓

LZW

Od LZ78 k LZW

Od LZ78 k LZW

LZ78 výstup pro ABABAB:

Od LZ78 k LZW

LZ78 výstup pro ABABAB: $(0, A)$ $(0, B)$ $(1, B)$ $(3, \$)$ – 4 dvojice

Od LZ78 k LZW

LZ78 výstup pro ABABAB: $(0, A)$ $(0, B)$ $(1, B)$ $(3, \$)$ – 4 dvojice

Každá dvojice obsahuje **index + char**

Od LZ78 k LZW

LZ78 výstup pro ABABAB: (0,A) (0,B) (1,B) (3,\$) – 4 dvojice

Každá dvojice obsahuje **index + char** – ale ten znak bude ve slovníku tak jako tak, proč ho zapisovat zvlášť?

Od LZ78 k LZW

LZ78 výstup pro ABABAB: (0,A) (0,B) (1,B) (3,\$) – 4 dvojice

Každá dvojice obsahuje **index + char** – ale ten znak bude ve slovníku tak jako tak, proč ho zapisovat zvlášť?

Terry Welch (1984) navrhl vylepšení – **LZW**:

Od LZ78 k LZW

LZ78 výstup pro ABABAB: (0,A) (0,B) (1,B) (3,\$) – 4 dvojice

Každá dvojice obsahuje **index + char** – ale ten znak bude ve slovníku tak jako tak, proč ho zapisovat zvlášť?

Terry Welch (1984) navrhl vylepšení – **LZW**:

- Slovník **předem obsahuje** všechny jednotlivé znaky

Od LZ78 k LZW

LZ78 výstup pro ABABAB: (0,A) (0,B) (1,B) (3,\$) – 4 dvojice

Každá dvojice obsahuje **index + char** – ale ten znak bude ve slovníku tak jako tak, proč ho zapisovat zvlášť?

Terry Welch (1984) navrhl vylepšení – **LZW**:

- Slovník **předem obsahuje** všechny jednotlivé znaky
- Na výstupu jsou **pouze kódy** (žádné dvojice)

Od LZ78 k LZW

LZ78 výstup pro ABABAB: (0,A) (0,B) (1,B) (3,\$) – 4 dvojice

Každá dvojice obsahuje **index + char** – ale ten znak bude ve slovníku tak jako tak, proč ho zapisovat zvlášť?

Terry Welch (1984) navrhl vylepšení – **LZW**:

- Slovník **předem obsahuje** všechny jednotlivé znaky
- Na výstupu jsou **pouze kódy** (žádné dvojice)

LZW výstup pro ABABAB:

Od LZ78 k LZW

LZ78 výstup pro ABABAB: (0,A) (0,B) (1,B) (3,\$) – 4 dvojice

Každá dvojice obsahuje **index + char** – ale ten znak bude ve slovníku tak jako tak, proč ho zapisovat zvlášť?

Terry Welch (1984) navrhl vylepšení – **LZW**:

- Slovník **předem obsahuje** všechny jednotlivé znaky
- Na výstupu jsou **pouze kódy** (žádné dvojice)

LZW výstup pro ABABAB: 1, 2, 3, 3 – 4 kódy (bez znaků navíc)

Kódování LZW

Kódování LZW

Počáteční slovník: každý jednotlivý znak má svůj kód

Kódování LZW

Počáteční slovník: každý jednotlivý znak má svůj kód

Algoritmus (w = aktuální sekvence, c = další znak):

Kódování LZW

Počáteční slovník: každý jednotlivý znak má svůj kód

Algoritmus (w = aktuální sekvence, c = další znak):

1. Načti znak c , přidej ho k w

Kódování LZW

Počáteční slovník: každý jednotlivý znak má svůj kód

Algoritmus (w = aktuální sekvence, c = další znak):

1. Načti znak c , přidej ho k w
2. Pokud $w + c$ **je** ve slovníku $\rightarrow$

Kódování LZW

Počáteční slovník: každý jednotlivý znak má svůj kód

Algoritmus (w = aktuální sekvence, c = další znak):

1. Načti znak c , přidej ho k w
2. Pokud $w + c$ **je** ve slovníku $\rightarrow$ prodluž w

Kódování LZW

Počáteční slovník: každý jednotlivý znak má svůj kód

Algoritmus (w = aktuální sekvence, c = další znak):

1. Načti znak c , přidej ho k w
2. Pokud $w + c$ **je** ve slovníku $\rightarrow$ prodluž w
3. Pokud $w + c$ **není** ve slovníku:

Kódování LZW

Počáteční slovník: každý jednotlivý znak má svůj kód

Algoritmus (w = aktuální sekvence, c = další znak):

1. Načti znak c , přidej ho k w
2. Pokud $w + c$ **je** ve slovníku $\rightarrow$ prodluž w
3. Pokud $w + c$ **není** ve slovníku:
 - Vypiš kód pro w

Kódování LZW

Počáteční slovník: každý jednotlivý znak má svůj kód

Algoritmus (w = aktuální sekvence, c = další znak):

1. Načti znak c , přidej ho k w
2. Pokud $w + c$ **je** ve slovníku $\rightarrow$ prodluž w
3. Pokud $w + c$ **není** ve slovníku:
 - Vypiš kód pro w
 - Přidej $w + c$ do slovníku

Kódování LZW

Počáteční slovník: každý jednotlivý znak má svůj kód

Algoritmus (w = aktuální sekvence, c = další znak):

1. Načti znak c , přidej ho k w
2. Pokud $w + c$ **je** ve slovníku $\rightarrow$ prodluž w
3. Pokud $w + c$ **není** ve slovníku:
 - Vypiš kód pro w
 - Přidej $w + c$ do slovníku
 - Začni novou sekvencí: $w = c$

Kódování LZW

Zakódujme ABABAB:

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB				

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A			

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B		

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB				

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B			

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A		

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA
3	ABAB				

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA
3	ABAB	AB			

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA
3	ABAB	AB	A		

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA
3	ABAB	AB	A	3	

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA
3	ABAB	AB	A	3	přidej 5 = ABA

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA
3	ABAB	AB	A	3	přidej 5 = ABA

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA
3	ABAB	AB	A	3	přidej 5 = ABA
4	AB				

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA
3	ABAB	AB	A	3	přidej 5 = ABA
4	AB	AB			

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA
3	ABAB	AB	A	3	přidej 5 = ABA
4	AB	AB	\$		

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA
3	ABAB	AB	A	3	přidej 5 = ABA
4	AB	AB	\$	3	

Kódování LZW

Zakódujme ABABAB:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABAB	A	B	1	přidej 3 = AB
2	BABAB	B	A	2	přidej 4 = BA
3	ABAB	AB	A	3	přidej 5 = ABA
4	AB	AB	\$	3	–

Vstup: 6 znaků → výstup: 4 kódy (1, 2, 3, 3)

Dekódování LZW

Dekódování LZW

Klíčová vlastnost: dekodér si buduje **stejný slovník** jako kodér

Dekódování LZW

Klíčová vlastnost: dekodér si buduje **stejný slovník** jako kodér – nepotřebuje ho přijmout

Dekódování LZW

Klíčová vlastnost: dekodér si buduje **stejný slovník** jako kodér – nepotřebuje ho přijmout

Algoritmus:

Dekódování LZW

Klíčová vlastnost: dekodér si buduje **stejný slovník** jako kodér – nepotřebuje ho přijmout

Algoritmus:

1. Přečti kód, najdi odpovídající sekvenci ve slovníku

Dekódování LZW

Klíčová vlastnost: dekodér si buduje **stejný slovník** jako kodér – nepotřebuje ho přijmout

Algoritmus:

1. Přečti kód, najdi odpovídající sekvenci ve slovníku
2. Přidej do slovníku: **předchozí sekvence + první znak aktuální**

Dekódování LZW

Klíčová vlastnost: dekodér si buduje **stejný slovník** jako kodér – nepotřebuje ho přijmout

Algoritmus:

1. Přečti kód, najdi odpovídající sekvenci ve slovníku
2. Přidej do slovníku: **předchozí sekvence + první znak aktuální**
3. Vypiš aktuální sekvenci

Dekódování LZW

Klíčová vlastnost: dekodér si buduje **stejný slovník** jako kodér – nepotřebuje ho přijmout

Algoritmus:

1. Přečti kód, najdi odpovídající sekvenci ve slovníku
2. Přidej do slovníku: **předchozí sekvence + první znak aktuální**
3. Vypiš aktuální sekvenci
4. Opakuj

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1			

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A		

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2			

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B		

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3			

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB		

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB	přidej 4 = BA	ABAB

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB	přidej 4 = BA	ABAB

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB	přidej 4 = BA	ABAB
4	3			

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB	přidej 4 = BA	ABAB
4	3	AB		

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB	přidej 4 = BA	ABAB
4	3	AB	přidej 5 = ABA	ABABAB

Dekódování LZW

Dekódujme 1, 2, 3, 3:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB	přidej 4 = BA	ABAB
4	3	AB	přidej 5 = ABA	ABABAB

Výsledek: ABABAB ✓

Speciální případ dekódování

Co když dekodér přečte kód, který ještě nemá ve slovníku?

Speciální případ dekódování

Co když dekodér přečte kód, který ještě nemá ve slovníku?

Zakódujme AAA:

Speciální případ dekódování

Co když dekodér přečte kód, který ještě nemá ve slovníku?

Zakódujme AAA:

Slovník:

Kód	Záznam
1	A

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	AAA				

Speciální případ dekódování

Co když dekodér přečte kód, který ještě nemá ve slovníku?

Zakódujme AAA:

Slovník:

Kód	Záznam
1	A

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	AAA	A			

Speciální případ dekódování

Co když dekodér přečte kód, který ještě nemá ve slovníku?

Zakódujme AAA:

Slovník:

Kód	Záznam
1	A

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	AAA	A	A	1	

Speciální případ dekódování

Co když dekodér přečte kód, který ještě nemá ve slovníku?

Zakódujme AAA:

Slovník:

Kód	Záznam
1	A

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	AAA	A	A	1	přidej 2 = AA

Speciální případ dekódování

Co když dekodér přečte kód, který ještě nemá ve slovníku?

Zakódujme AAA:

Slovník:

Kód	Záznam
1	A
2	AA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	AAA	A	A	1	přidej 2 = AA

Speciální případ dekódování

Co když dekodér přečte kód, který ještě nemá ve slovníku?

Zakódujme AAA:

Slovník:

Kód	Záznam
1	A
2	AA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	AAA	A	A	1	přidej 2 = AA
2	AA				

Speciální případ dekódování

Co když dekodér přečte kód, který ještě nemá ve slovníku?

Zakódujme AAA:

Slovník:

Kód	Záznam
1	A
2	AA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	AAA	A	A	1	přidej 2 = AA
2	AA	AA			

Speciální případ dekódování

Co když dekodér přečte kód, který ještě nemá ve slovníku?

Zakódujme AAA:

Slovník:

Kód	Záznam
1	A
2	AA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	AAA	A	A	1	přidej 2 = AA
2	AA	AA	\$	2	–

Výstup kodéru: 1, 2 – kodér **právě přidal** záznam 2 = AA a **ihned** ho použil

Speciální případ dekódování

Dekódujme 1, 2:

Speciální případ dekódování

Dekódujme 1, 2:

Slovník:

Kód	Záznam
1	A

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1			

Speciální případ dekódování

Dekódujme 1, 2:

Slovník:

Kód	Záznam
1	A

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A		

Speciální případ dekódování

Dekódujme 1, 2:

Slovník:

Kód	Záznam
1	A

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A

Speciální případ dekódování

Dekódujme 1, 2:

Slovník:

Kód	Záznam
1	A

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	není ve slovníku!		

Speciální případ dekódování

Dekódujme 1, 2:

Slovník:

Kód	Záznam
1	A
2	AA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	není ve slovníku!	přidej 2 = AA	

Speciální případ dekódování

Dekódujme 1, 2:

Slovník:

Kód	Záznam
1	A
2	AA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	není ve slovníku!	přidej 2 = AA	AAA

Řešení:

Nová sekvence = předchozí + první znak předchozí = „A“ + „A“ = „AA“

Tento případ nastane, když kodér **přidal** záznam a **ihned** ho použil

Speciální případ dekódování

Proč to funguje?

Speciální případ dekódování

Proč to funguje?

Kodér v kroku 1 vypsál „A“, podíval se na další znak c a přidal záznam
 $2 = \text{„A“} + c$

Speciální případ dekódování

Proč to funguje?

Kodér v kroku 1 vypsál „A“, podíval se na další znak c a přidal záznam $2 = \text{„A“} + c$

Potom kodér pokračoval **od znaku c** – takže cokoliv pošle příště, **začíná znakem c**

Speciální případ dekódování

Proč to funguje?

Kodér v kroku 1 vypsál „A“, podíval se na další znak c a přidal záznam $2 = \text{„A“} + c$

Potom kodér pokračoval **od znaku c** – takže cokoliv pošle příště, **začíná znakem c**

A co poslal příště?

Speciální případ dekódování

Proč to funguje?

Kodér v kroku 1 vypsál „A“, podíval se na další znak c a přidal záznam $2 = „A“ + c$

Potom kodér pokračoval **od znaku c** – takže cokoliv pošle příště, **začíná znkem c**

A co poslal příště? Kód 2 , tedy záznam „A“ + c – ten ale **začíná na „A“**

Speciální případ dekódování

Proč to funguje?

Kodér v kroku 1 vypsál „A“, podíval se na další znak c a přidal záznam $2 = „A“ + c$

Potom kodér pokračoval **od znaku c** – takže cokoliv pošle příště, **začíná znkem c**

A co poslal příště? Kód 2 , tedy záznam „A“ + c – ten ale **začíná na „A“**

Takže výstup začíná na c a zároveň na „A“

Speciální případ dekódování

Proč to funguje?

Kodér v kroku 1 vypsál „A“, podíval se na další znak c a přidal záznam $2 = „A“ + c$

Potom kodér pokračoval **od znaku c** – takže cokoliv pošle příště, **začíná znakem c**

A co poslal příště? Kód 2 , tedy záznam „A“ + c – ten ale **začíná na „A“**

Takže výstup začíná na c a zároveň na „A“ – to může platit jedině když $c = „A“$

Speciální případ dekódování

Proč to funguje?

Kodér v kroku 1 vypsál „A“, podíval se na další znak c a přidal záznam $2 = „A“ + c$

Potom kodér pokračoval **od znaku c** – takže cokoliv pošle příště, **začíná znakem c**

A co poslal příště? Kód **2**, tedy záznam „A“ + c – ten ale **začíná na „A“**

Takže výstup začíná na c a zároveň na „A“ – to může platit jedině když $c = „A“$ a tedy záznam $2 = „A“ + „A“ = „AA“$

Speciální případ dekódování

A co když je předchozí výstup delší?

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA				

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A			

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA				

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B			

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B	A	2	

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B	A	2	přidej 4 = BA

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B	A	2	přidej 4 = BA

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B	A	2	přidej 4 = BA
3	ABABA				

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B	A	2	přidej 4 = BA
3	ABABA	AB			

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B	A	2	přidej 4 = BA
3	ABABA	AB	A	3	

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B	A	2	přidej 4 = BA
3	ABABA	AB	A	3	přidej 5 = ABA

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B	A	2	přidej 4 = BA
3	ABABA	AB	A	3	přidej 5 = ABA

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B	A	2	přidej 4 = BA
3	ABABA	AB	A	3	přidej 5 = ABA
4	ABA				

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B	A	2	přidej 4 = BA
3	ABABA	AB	A	3	přidej 5 = ABA
4	ABA	ABA			

Speciální případ dekódování

A co když je předchozí výstup delší? Zakódujme ABABABA:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Kódování:

Krok	Vstup	Sekvence	Další	Výstup	Akce
1	ABABABA	A	B	1	přidej 3 = AB
2	BABABA	B	A	2	přidej 4 = BA
3	ABABA	AB	A	3	přidej 5 = ABA
4	ABA	ABA	\$	5	–

Kodér v kroku 3 přidal **5 = ABA** a v kroku 4 ho **ihned použil**

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Slovník:

Kód	Záznam
1	A
2	B

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1			

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Slovník:

Kód	Záznam
1	A
2	B

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A		

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Slovník:

Kód	Záznam
1	A
2	B

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Slovník:

Kód	Záznam
1	A
2	B
3	AB

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB		

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB	přidej 4 = BA	

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB	přidej 4 = BA	ABAB

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB	přidej 4 = BA	ABAB
4	5	není ve slovníku!		

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB	přidej 4 = BA	ABAB
4	5	není ve slovníku!	přidej 5 = ABA	

Speciální případ dekódování

Dekódujme 1, 2, 3, 5:

Slovník:

Kód	Záznam
1	A
2	B
3	AB
4	BA
5	ABA

Dekódování:

Krok	Kód	Sekvence	Akce	Výstup
1	1	A	–	A
2	2	B	přidej 3 = AB	AB
3	3	AB	přidej 4 = BA	ABAB
4	5	není ve slovníku!	přidej 5 = ABA	ABABABA

Speciální případ dekódování

Předchozí výstup = „AB“ $\rightarrow$ nový záznam = „AB“ + první znak „AB“ = „ABA“

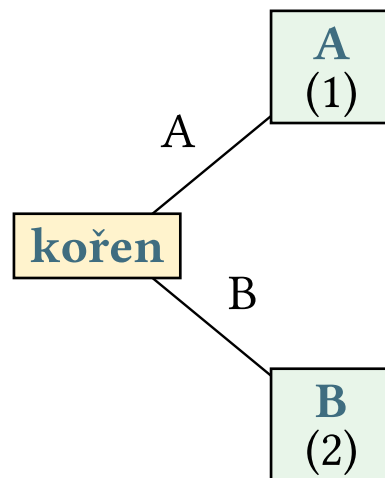
U delšího prefixu je vidět, že se bere **první znak** celé předchozí sekvence

Slovník jako strom

Slovník LZW má strukturu **prefixového stromu**
(*trie* – z angl. re**trie**val)

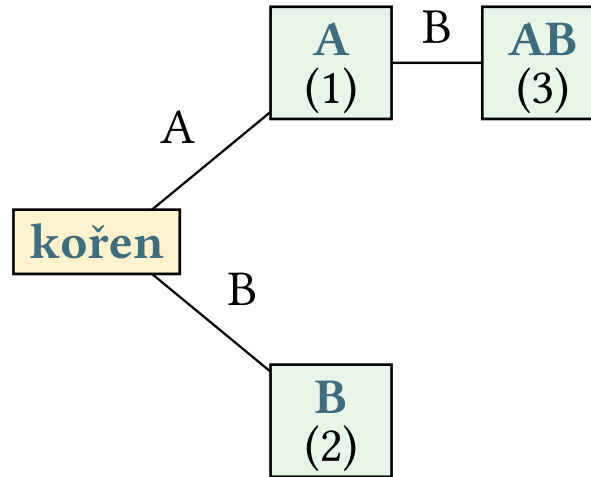
Slovník jako strom

Počáteční stav:



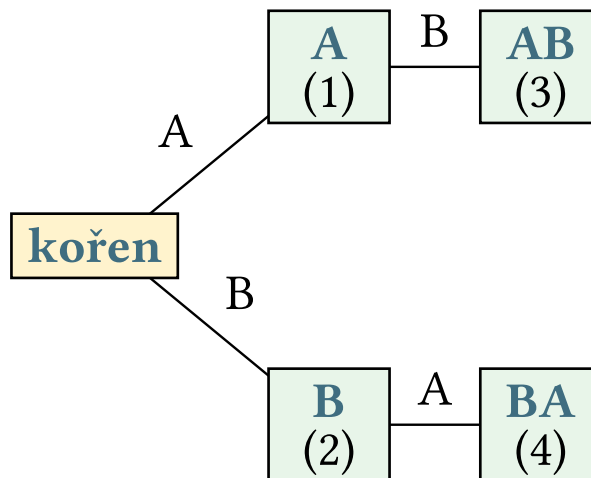
Slovník jako strom

Krok 1: výstup 1 (A), přidáme $AB = 3$:



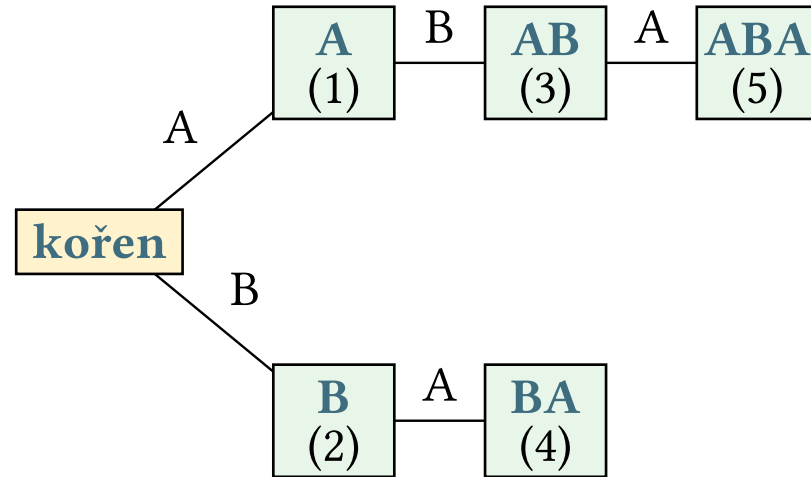
Slovník jako strom

Krok 2: výstup 2 (B), přidáme $BA = 4$:



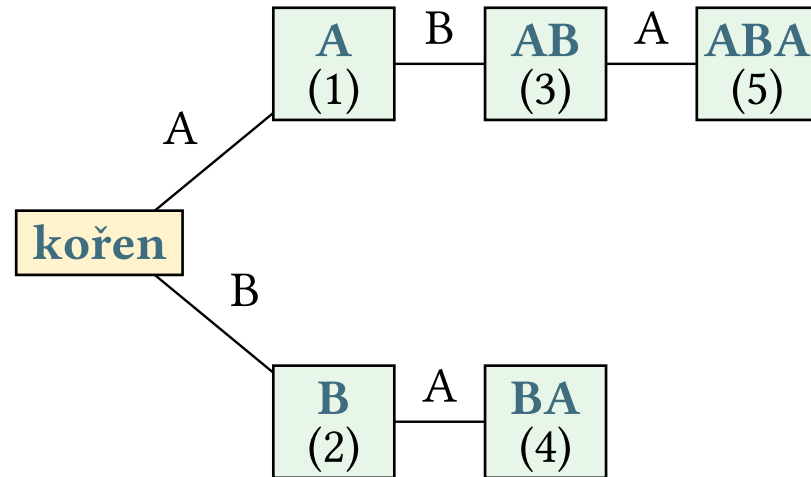
Slovník jako strom

Krok 3: výstup 3 (AB), přidáme $ABA = 5$:



Slovník jako strom

Krok 3: výstup 3 (AB), přidáme ABA = 5:



Kódování hledá **nejdelší cestu** od kořene, která odpovídá vstupu

Správa slovníku

Co když slovník příliš naroste?

Správa slovníku

Co když slovník příliš naroste?

Kódy ve slovníku jsou binární čísla

Správa slovníku

Co když slovník příliš naroste?

Kódy ve slovníku jsou binární čísla

Pro 4096 záznamů potřebujeme 12 bitů na kód (GIF standard)

Správa slovníku

Co když slovník příliš naroste?

Kódy ve slovníku jsou binární čísla

Pro 4096 záznamů potřebujeme 12 bitů na kód (GIF standard)

Dvě strategie:

Správa slovníku

Co když slovník příliš naroste?

Kódy ve slovníku jsou binární čísla

Pro 4096 záznamů potřebujeme 12 bitů na kód (GIF standard)

Dvě strategie:

- **Zmrazení** – přestaň přidávat nové záznamy

Správa slovníku

Co když slovník příliš naroste?

Kódy ve slovníku jsou binární čísla

Pro 4096 záznamů potřebujeme 12 bitů na kód (GIF standard)

Dvě strategie:

- **Zmrazení** – přestaň přidávat nové záznamy
- **Reset** – vymaž slovník a začni znovu

Správa slovníku

Co když slovník příliš naroste?

Kódy ve slovníku jsou binární čísla

Pro 4096 záznamů potřebujeme 12 bitů na kód (GIF standard)

Dvě strategie:

- **Zmrazení** – přestaň přidávat nové záznamy
- **Reset** – vymaž slovník a začni znovu (GIF - speciální „clear code“)

Správa slovníku

Co když slovník příliš naroste?

Kódy ve slovníku jsou binární čísla

Pro 4096 záznamů potřebujeme 12 bitů na kód (GIF standard)

Dvě strategie:

- **Zmrazení** – přestaň přidávat nové záznamy
- **Reset** – vymaž slovník a začni znovu (GIF - speciální „clear code“)

Některé implementace používají **variabilní šířku** kódů

Správa slovníku

Co když slovník příliš naroste?

Kódy ve slovníku jsou binární čísla

Pro 4096 záznamů potřebujeme 12 bitů na kód (GIF standard)

Dvě strategie:

- **Zmrazení** – přestaň přidávat nové záznamy
- **Reset** – vymaž slovník a začni znovu (GIF - speciální „clear code“)

Některé implementace používají **variabilní šířku** kódů – začínají s 9 bity a postupně rozšiřují

Kde se LZW používá?

Kde se LZW používá?

- GIF

Kde se LZW používá?

- **GIF** – formát obrázků (max 256 barev)

Kde se LZW používá?

- **GIF** – formát obrázků (max 256 barev)
- **TIFF**

Kde se LZW používá?

- **GIF** – formát obrázků (max 256 barev)
- **TIFF** – volitelná komprese v TIFF souborech

Kde se LZW používá?

- **GIF** – formát obrázků (max 256 barev)
- **TIFF** – volitelná komprese v TIFF souborech
- **PDF**

Kde se LZW používá?

- **GIF** – formát obrázků (max 256 barev)
- **TIFF** – volitelná komprese v TIFF souborech
- **PDF** – jedna z podporovaných kompresních metod

Kde se LZW používá?

- **GIF** – formát obrázků (max 256 barev)
- **TIFF** – volitelná komprese v TIFF souborech
- **PDF** – jedna z podporovaných kompresních metod
- unixový příkaz `compress`

Kde se LZW používá?

- **GIF** – formát obrázků (max 256 barev)
- **TIFF** – volitelná komprese v TIFF souborech
- **PDF** – jedna z podporovaných kompresních metod
- unixový příkaz compress (dnes nahrazen gzip)

Kde se LZW používá?

- **GIF** – formát obrázků (max 256 barev)
- **TIFF** – volitelná komprese v TIFF souborech
- **PDF** – jedna z podporovaných kompresních metod
- unixový příkaz compress (dnes nahrazen gzip)

LZW byl historicky **patentovaný** (patent Unisys)

Kde se LZW používá?

- **GIF** – formát obrázků (max 256 barev)
- **TIFF** – volitelná komprese v TIFF souborech
- **PDF** – jedna z podporovaných kompresních metod
- unixový příkaz compress (dnes nahrazen gzip)

LZW byl historicky **patentovaný** (patent Unisys) – to vedlo k vývoji alternativ (PNG místo GIF)

Kde se LZW používá?

- **GIF** – formát obrázků (max 256 barev)
- **TIFF** – volitelná komprese v TIFF souborech
- **PDF** – jedna z podporovaných kompresních metod
- unixový příkaz compress (dnes nahrazen gzip)

LZW byl historicky **patentovaný** (patent Unisys) – to vedlo k vývoji alternativ (PNG místo GIF) a posílilo odpor proti **softwarovým patentům**

Shrnutí

Srovnání metod

	LZ77	LZ78	LZW
Rok	1977	1978	1984
Slovník	posuvné okno	explicitní	explicitní + předvyplněný
Výstup	(offset, length, char)	(index, char)	(code)
Dekodér	sleduje reference	buduje slovník	buduje slovník
Použití	gzip, zstd, deflate	–	GIF, TIFF, PDF

Shrnutí

Shrnutí

- **Slovníkové metody** nahrazují opakující se sekvence **referencemi**

Shrnutí

- **Slovníkové metody** nahrazují opakující se sekvence **referencemi**
- **LZ77** – posuvné okno, trojice (offset, length, next_char), základ deflate

Shrnutí

- **Slovníkové metody** nahrazují opakující se sekvence **referencemi**
- **LZ77** – posuvné okno, trojice (offset, length, next_char), základ deflate
- **LZ78** – explicitní slovník, dvojice (index, char)

Shrnutí

- **Slovníkové metody** nahrazují opakující se sekvence **referencemi**
- **LZ77** – posuvné okno, trojice (offset, length, next_char), základ deflate
- **LZ78** – explicitní slovník, dvojice (index, char)
- **LZW** – vylepšení LZ78, výstup (code), dekodér si slovník buduje sám (GIF, TIFF)

Shrnutí

- **Slovníkové metody** nahrazují opakující se sekvence **referencemi**
- **LZ77** – posuvné okno, trojice (offset, length, next_char), základ deflate
- **LZ78** – explicitní slovník, dvojice (index, char)
- **LZW** – vylepšení LZ78, výstup (code), dekodér si slovník buduje sám (GIF, TIFF)
- V praxi se metody **kombinují** – každá řeší jinou část problému